



CAGTRADE: Predicting Stock Market Price Movement with a CNN-Attention-GRU Model

Ibanga Kpereobong Friday¹ · Sarada Prasanna Pati¹ · Debahuti Mishra¹ · Pradeep Kumar Mallick² · Sachin Kumar³ 

Accepted: 25 April 2024 / Published online: 17 June 2024

© The Author(s), under exclusive licence to Springer Japan KK, part of Springer Nature 2024

Abstract

Accurately predicting market direction is crucial for informed trading decisions to buy or sell stocks. This study proposes a deep learning based hybrid approach combining convolutional neural network (CNN), attention mechanism (AM), and gated recurrent unit (GRU) to predict short-term market trends (1 day, 3 days, 7 days, 10 days) across different stock indices (BSE, HSI, IXIC, NIFTY, N225, SSE). The architecture dynamically weights the input sequence with the AM model, captures local patterns through CNN and effectively models long-term dependencies with GRU thus aiming to accurately classify either "*buy*" or "*sell*" positions of stocks. The model is assessed using classification and financial evaluation metrics involving accuracy, precision, recall, f1-score, annualized returns, maximum drawdown, and return on investment. It outperforms benchmark models, and different technical indicators including average directional index, rate of change, moving average convergence divergence, and the buy-and-hold strategy, demonstrating its effectiveness in various market conditions. The proposed model achieves an average accuracy of 98% in predicting the 1 day-ahead direction, and an average accuracy of 88.53% across all prediction intervals. The model was also validated using the wilcoxon signed rank test that further supported its significance over the benchmark models. The CAG model presents a comprehensive and intuitive approach to stock market trend prediction, with potential applications in real-world asset decision-making.

Keywords Convolutional neural network (CNN) · Deep learning (DL) · Gated recurrent unit (GRU) · Imbalanced dataset · Attention mechanism (AM) · Stock market trend prediction

1 Introduction

Stock market prediction is a long-standing challenge, as it is a volatile and dynamic field that involves various disciplines, such as finance, economics, accounting, computer science, etc. (Fischer & Krauss, 2018; Gandhmal & Kumar, 2019; Oscar & Pomares-Quimbaya, 2020; Vargas et al., 2017). The question of whether stock markets can be accurately predicted and exploited by applying statistical and econometric methodologies has piqued significant interest in academic and professional groups (Engle, 2001; Gandhmal & Kumar, 2019). However, several of such existing models have shown limitations in terms of predicting accurate market trends over a long period (Banik et al., 2022; Chatzis et al., 2018). In recent years, there has been a growing focus on leveraging the capabilities of deep learning (DL) techniques across different applications from weather prediction (Hu et al., 2016), image classification (Chen et al., 2014), and text generation (Fogel et al., 2020), among others. These DL-based models have emerged as a formidable tool for predictive modeling due to their ability to deal with non-linear data (Istiake Sunny & M. M. S. Maswood A. G. Alharbi, 2020) and extract meaningful patterns (Terrence et al., 2011), as evidenced by the study conducted by Boming et al. (Huang et al., 2019). This paradigm shift has challenged traditional economic theories, such as the parsimony theory proposed by Elliot Sober (Sober, 1981), favoring models with fewer parameters. It has also challenged the long-standing random walk theory (James & V. H. et al., 1967), which posits that the stock market is highly random and cannot be reliably exploited, and Malkiel Burton's efficient market hypothesis theory (EMH) (Malkiel & B., 2003), which suggests that markets have efficient information that ensures it cannot be frequently outperformed. Extensive research has shown the efficacy of these DL-based models with promising approaches to accurately predict market movements which can be traced to their ability to handle and identify intricate data patterns and dependencies with the potential to be improved (Lei et al., 2020).

Short-term or swing trading (Banik et al., 2022) is a popular strategy employed by different market participants, which includes buying stocks within a relatively short time frame, typically spanning days to weeks. The primary objective of this type of trading is to maximize profits by capitalizing on favorable market trends, which involves buying stocks during bullish (rising) periods and selling them during bearish (declining) periods. This strategy heavily relies on analyzing price movements and trends, with stock market experts utilizing different technical indicators to identify the most advantageous entry and exit points for their trades. This trading approach has led to an increase in implementing and training DL techniques such as long short-term memory (LSTM), attention mechanism (AM), reinforcement learning (RL), convolutional neural network (CNN), and gated recurrent unit (GRU), among others.

Several studies, including those by Kai et al. (Lei et al., 2020) and Pouya, Akbar & Hassan (Khodaei et al., 2022), have demonstrated the effectiveness of hybrid models in achieving improved performance. These models combine the predictive abilities of individual models, such as CNNs for local feature

extraction, AM for understanding relationships between input features, and recurrent neural networks (RNNs) like LSTMs and GRUs for processing long-term dependencies in sequential data. Shicheng Li et al. (Li et al., 2023) proposed the Autoencoder-AM-CNN-GRU (AE-ACG) model, which processes weighted input data through an AM block and then extracts features and predicts the closing price using a CNN-GRU block. This model also employs a skip connection to bridge the encoder and decoder parts of the autoencoder, achieving the lowest error rate for the next-day close price prediction across all the metrics taken. Similarly, Yang et al. (Yang et al., 2022) proposed the CNN-GRUA-FC model, using Random Forest to select important features and feeding them into separate CNN and GRU models with an AM model. The CNN-GRUA-FC model achieved an R^2 score of 0.9108 for next-day close price prediction. This hybrid model architecture demonstrates its effectiveness by leveraging the strengths of individual models by distributing the tasks optimally, enabling each model to contribute its best capabilities without compromising prediction accuracy.

Inspired by the effectiveness of the hybrid architecture, we propose a similar approach, the CNN-AM-GRU (CAG) model, for accurate short-term market trend prediction across various intervals (1 day, 3 days, 7 days, and 10 days). Jingyi & M. Omair (Shen & Shafiq, 2020), among others, exemplify this approach of classifying market movements into buy and sell labels. The target variable is determined by the market's upward or downward movement, calculated as the price change between the current close price and the close price n -trading days later. Unlike the studies (Li et al., 2023; Yang et al., 2022) that focused on regression for the next day closing price prediction, this work addresses market movement classification, a different problem requiring the classification of buy or sell positions over different timeframes.

The key contributions of this study are:

1. The AM dynamically weighs and analyzes the input sequence, comparing each element and generating weights based on their relative importance. This allows the model to focus on informative features, even in the minority class, leading to better classification performance.
2. The CNN model applies learned filters to the weighted input sequence to capture local patterns and extract features focusing on the short-term patterns within the input sequence.
3. The GRU model effectively captures sequential dependencies and long-term patterns present in the data.
4. The proposed model significantly expands the prediction horizon beyond the 1 day-ahead forecasts typical in similar studies. It predicts upward or downward market trends across different short-term intervals.
5. The performance of the proposed model is evaluated using classification metrics, including accuracy, precision, recall, and f1-score. Additionally, the results of the predicted label generated by the model were employed to simulate transactions using a trading strategy depicting a real-world financial market scenario.

The following sections of this study are organized thus: following this concise introduction in Sect. 1, Sect. 2 reviews relevant studies that have influenced this work, followed by a detailed discussion of the proposed model and methodology in Sect. 3. Section 4 provides the data collection and preprocessing steps used in this work. Section 5 presents the experimental results, incorporating commonly used performance metrics and profitability simulations comparing our model to benchmark models. Finally, Sect. 6 concludes the paper by summarizing the findings and proposing future research directions.

2 Literature Survey

Stock market prediction has garnered significant attention in recent years, with a particular focus on developing models capable of accurately identifying upward and downward movements in stock prices. However, advancements in DL have led to the development of innovative models that leverage image data derived from historical price data, along with the integration of technical indicators, to improve prediction accuracy.

A pioneering study by Sezer and Ozbayoglu (Sezer & Ozbayoglu, 2018) explored the utilization of a 2-D CNN model. They converted time series data into 2-D images and incorporated 15 distinct technical indicators for analysis. The study evaluated their model's performance against other comparable models using Dow-30 stocks and exchange-traded funds (ETFs). However, a limitation of this study was that only peak points were labeled, which restricts the potential profitability and may not adequately address trading range markets characterized by sideways price movements without clear trends. In a similar vein, Hyun Sik Sim et al. (Sim et al., 2019) adopted a CNN model for stock market prediction by transforming historical data into images depicting time series graphs, akin to the methodology employed by Sezer and Ozbayoglu (2018). They compared the performance of their optimized CNN model against alternative models in their study. In a different direction, Suhwan Ji et al. (Ji et al., 2019) utilized LSTM and DNN models, and the use of a correlation coefficient to remove features not closely related to price serves as an excellent approach to reducing redundant features even when using a DL model. Another study focused on hybrid models, such as the one proposed by Pouya Khodaei et al. (Khodaei et al., 2022) proposed a hybrid CNN-LSTM-ResNet model that sought to address the issue of imbalanced classes in the training process. They employed a balancing approach to ensure an equal number of instances for each class, rather than dealing with three unbalanced classes (*Buy*, *Hold*, and *Sell*). Significant contributions have been made by the ensemble LSTM approach proposed by Svetlana Borovkova et al. (Borovkova & Tsiamas, 2019). Their methodology involved classifying price movements into three distinct categories: strong sell, hold, and strong buy. This approach demonstrated the potential of employing LSTM models in capturing and categorizing market dynamics. Wenjie Lu et al. (Wenjie et al., 2021) propose a hybrid model that combines CNN, bidirectional LSTM (BiLSTM)—a variant of the LSTM model—and AM to forecast the price for the following day. The methodology in this study involves using the CNN model to extract features from input

data. The Bi-LSTM layer then employs these features for future predictions, while the AM is employed to fine-tune feature states and enhance prediction outcomes. Although this proposed approach outperforms other benchmark models utilized in the study, its limitation lies in predicting the close price of only 1 day. This temporal range might not be sufficient for many market participants to make well-informed and adequate decisions. This work addresses this limitation by attempting to classify short-term price movements across various intervals (1 day, 3 days, 7 days, and 10 days), all while achieving a commendable level of predictive performance.

These myriad works exemplify a range of approaches and advancements in the utilization of DL models for stock market prediction. These approaches focus on different considerations of dealing with such data such as feature selection, integration of various technical indicators, and class balancing techniques, to improve the accuracy of the models.

3 Proposed Model & Methodology Used

In the pursuit for more accurate and robust stock market predictions, this study introduces a novel model that capitalizes on the individual strengths of the CNN, AM, and GRU models. The proposed model, known as the CAG model, aims to leverage the advantages of each component to enhance the predictive capabilities in the dynamic and volatile stock market landscape depicted in Fig. 1.

3.1 Attention Mechanism (AM)

The proposed model utilizes the AM initially introduced by Treisman et al. (Treisman & Gelade, 1980), later extended by Vaswani et al. (Vaswani et al., 2017), to focus on the most important parts of an input sequence. This mechanism ensures effective information extraction, optimizing sequential processing by allocating probability weights to focus on important information (Liu, 2018). In this context, attention scores are adjusted based on the class distribution in the trading data, leveraging the model to capture crucial patterns in financial

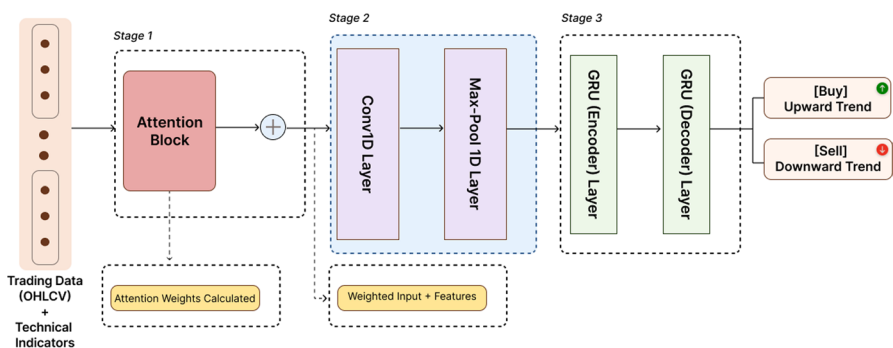


Fig. 1 The architecture of the CAG model

data. For instance, Liu Huicheng (Liu, 2018) proposed an attention-based cyclic neural network to train financial news data for predicting stock prices. The study concluded that by employing AM, the model learned to prioritize relevant information in the financial news, ultimately improving its predictive accuracy. Similarly, Shun & Lei (Shun Chen & Lei Ge, 2019) demonstrated the effectiveness of AM in an LSTM-based prediction method, where the hybrid model adequately outperformed the LSTM model and exhibited better profitability than benchmark models.

In this work, the input data is represented as a matrix X with dimensions (T, D) , where T is the sequence length, and D is the number of features. The input sequence X is passed through the AM that computes the attention weights, α , to emphasize or de-emphasize certain elements of the sequence based on their relevance. The attended features H_{attn} are obtained as the weighted sum of the input sequence:

$$Z = \tanh(W_{attn}.X.T) \quad (1)$$

$$\alpha = \text{softmax} U_{attn}.Z \quad (2)$$

$$H_{attn} = \alpha.T.x \quad (3)$$

where W_{attn} and U_{attn} are learnable weight matrices.

3.2 Convolutional Neural Network (CNN)

The CNN model was originally proposed by Yann Lecun et al. (LeCun & Bengio, 1995), and it plays a crucial role in extracting relevant features from different input data types (Kim & Kim, 2019; Sharma et al., 2023; Vargas et al., 2017). In this study, the CNN model was used to extract features from trading data sequences. The CNN model offers several advantages, such as capturing global spatial features, reducing the number of parameters and features, and mitigating overfitting as evidenced in Khodaei et al. (2022); Wenjie et al., 2021).

The attended features H_{attn} are passed through the CNN to extract hierarchical features. The CNN applies filters and pooling operations to capture relevant patterns in the attended features. Let's denote the output of the CNN as H_{CNN} with dimensions (T_{CNN}, F_{CNN}) , where T_{CNN} the new sequence length after CNN processing, F_{CNN} is the number of feature maps obtained from the CNN.

$$H_{CNN}[T, F] = \text{relu}(W_{CNN}[F].H_{attn}[T] + b_{CNN}[F]) \quad (4)$$

where $H_{CNN}[T, F]$ is the output of the CNN at time step t and filter f , $W_{CNN}[F]$ is the weight matrix of the f^{th} filter in the CNN layer, $b_{CNN}[F]$ is the bias vector of the f^{th} filter, $\text{relu}(x)$ is the *activation function* which applies an element-wise operation, returning the maximum of 0 and x .

3.3 Gated Recurrent Unit (GRU)

For learning long-term dependencies within the series, the proposed model employs the GRU model, originally proposed by Junyoung et al. (Chung et al., 2014). The output of the CNN, H_{CNN} , processing is passed through the GRU to model sequential dependencies and make the final prediction of the price movement of the respective interval.

The GRU processes the input sequence H_{CNN} , and at each time step, it produces a hidden state H_{GRU} . The hidden states are computed using the following equations:

$$R_T = \text{sigmoid}(W_R.H_{CNN}[T] + U_Z.H_{GRU}[T-1]) \quad (5)$$

$$Z_T = \text{sigmoid}(W_Z.H_{CNN}[T] + U_Z.H_{GRU}[T-1]) \quad (6)$$

$$H_T = \tanh W_H.H_{CNN}[T] + U_H.(R_T.H_{GRU}[T-1]) \quad (7)$$

$$H_{GRU}[T] = (1 - Z_T).H_T + Z_T.H_{GRU}[T - 1] \quad (8)$$

where R_T is the reset gate, Z_T is the update gate, H_T is the candidate hidden state, softmax is the activation function utilized, and $H_{GRU}[T]$ is the hidden state at time step t . The final output of the proposed model can be obtained by passing the last hidden state $H_{GRU}[T_{CNN}]$ through a fully connected layer with weights W_{OUT} :

$$\text{output} = \text{softmax}(W_{OUT}.H_{GRU}[T_{CNN}]) \quad (9)$$

3.4 Algorithm of the Cag Model

While the combination of CNN, GRU, and AM has been explored in prior studies following examples in Li et al. (2023); Yang et al., (2022), the approach is applied differently, predicting the market direction of each stock instead of the next day's close price. The CAG model demonstrates its potential to enhance decision-making in stock market prediction through a careful architecture design and model optimization. It efficiently captures all local and global features in each dataset, optimizes learning through attention-weighted processing, and handles long-term dependencies. Algorithm 1 summarizes the overview of the proposed CAG model. The data is collected, regenerated through the addition of technical indicators, and labeled with *buy* and *sell* labels—a comprehensive discussion of which is given in the ensuing section. Subsequently, this data is divided into training, validation, and test sets, used for both model training and the validation of its performance. The proposed CAG model's parameters are meticulously defined and trained with 50 epochs. The proficiently trained model is then used to predict market movements across the test datasets. The general framework of this methodology is given in Fig. 2.

Algorithm 1 Proposed CAG model

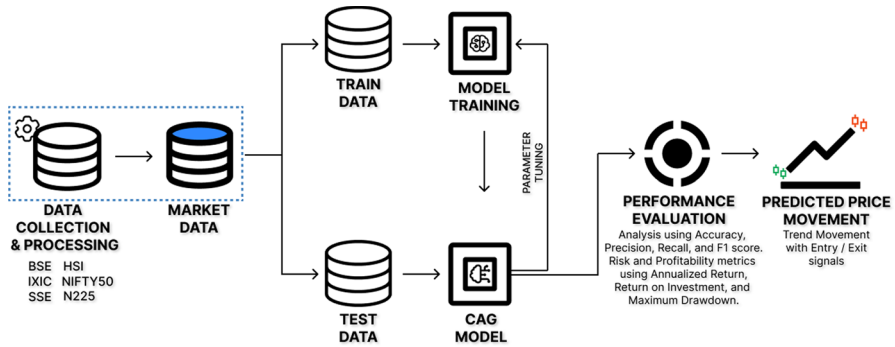


Fig. 2 General Framework of Proposed CAG model

Input: Training Data consisting of OHLCV data

Output: Short-term market movement across different intervals

```

1.      Begin
2.          read datasets consisting of open, close, high, low, volume, and adj close.
3.          dataset regeneration using technical indicators
4.          calculate labels (buy and sell labels)
5.          split the dataset into training, validation, and test sets:
6.              calculate the size of the training set: size = length(data) * 0.7
7.              assign the first 'size' rows to the training set: train = data[0:size]
8.              assign the next 'size' rows to the validation set: val = data[size:size+size]
9.              assign the remaining rows to the test set: test = data[size+size:end]
10.         return train, val, test
11.         compute label weights
12.         define the cag_model:
13.             create a sequential model:
14.                 add an attention layer
15.                 add a maxpooling1d layer (pool_size)
16.                 add a conv1d layer (filters, kernel_size, and activation_function)
17.                 add a gru layer (neurons, return_sequence=true)
18.                 add a flatten layer
19.                 add a dense layer with activation_function='sigmoid'
20.         return the cag_model
21.         train the cag_model:
22.             set the number of epochs: epoch = 50
23.             fit the model using the training data and validation data: model.fit(train_features,
24.                 train_labels, epochs=epoch, validation_data=(val_features, val_labels),
25.                 class_weights)
26.         make predictions using the trained model:
27.             use the trained model to predict the market movement on the test data: predictions =
28.                 model.predict(test_features)
29.             evaluate the model's performance:
30.                 convert the predicted probabilities into binary predictions: binary_predictions =
31.                     round(predictions)
32.                 calculate the accuracy score of the model: accuracy = accuracy_score(test_labels,
33.                     binary_predictions)
34.             print the accuracy score
35.         End

```


4 Data Collection and Pre-processing

This section provides a comprehensive overview of the key steps undertaken in this study to collect, preprocess, and label the data. The following subsections delve into the intricate details of each processing technique.

4.1 Data Collection

Different indices of historical data from various countries were used for the experiments undertaken in this study. All the datasets were collected from a popular financial data source, yahoo finance (YahooFinance. At <https://finance.yahoo.com>.CSV file. xxxx). The datasets encompassed a substantial timeframe, spanning from January 1, 2015, to December 31, 2022, providing robust and sufficient data for experiment and analysis. Each dataset comprised of essential trading information such as open, close, high, low, volume, and adjusted close values. We take 70% of the trading data for training, while the remaining 30% is used for validation and testing. Table 1 presents a comprehensive summary of each index data employed in this study. Figure 3 presents the price movements that visually represent the datasets used, by also providing valuable insights into its characteristics.

4.2 Feature Expansion by Using Technical Indicators

The market data was expanded to incorporate technical indicators to meet the research objective of effectively capturing short and long-term trends. By incorporating additional features, we were able to utilize the model's ability to identify patterns that would have been previously difficult to detect using market data alone. This study utilized 18 technical indicators from the market data, very similar to a previous study conducted by Smruti Rekha Das et al. (Rekha Das et al., 2019). The technical indicators used were: simple moving average (SMA), del c, exponential moving average (EMA), relative strength index (RSI), average true range (ATR), williams %r (W%), rate of change (ROC), stochastic oscillator (SO), average directional index (ADX), moving average convergence divergence (MACD), bollinger bands (BB), trix (TRIX), volatility index (VIX), commodity channel index (CCI), and on-balance-volume (OBV).

Table 1 Summary statistics of selected stocks

Index name	Description	Count	Mean	Max	Min	Standard deviation
BSE	Bombay Stock Exchange	2014	12845.46	17442.53	8664.94	2070.47
HSI	Hang Seng Index	1971	25143.53	33484.07	14597.01	3382.80
IXIC	NASDAQ-100 Index	2014	8543.09	16212.23	4209.75	3320.84
NIFTY 50	National Stock Exchange of India 50	1968	11586.82	18887.59	6825.79	3270.11
N225	Nikkei 225 Stock Average	1954	22390.28	30795.77	14864.00	3919.36
SSE	Shanghai Composite Index	1945	3216.56	5178.19	2440.90	368.80

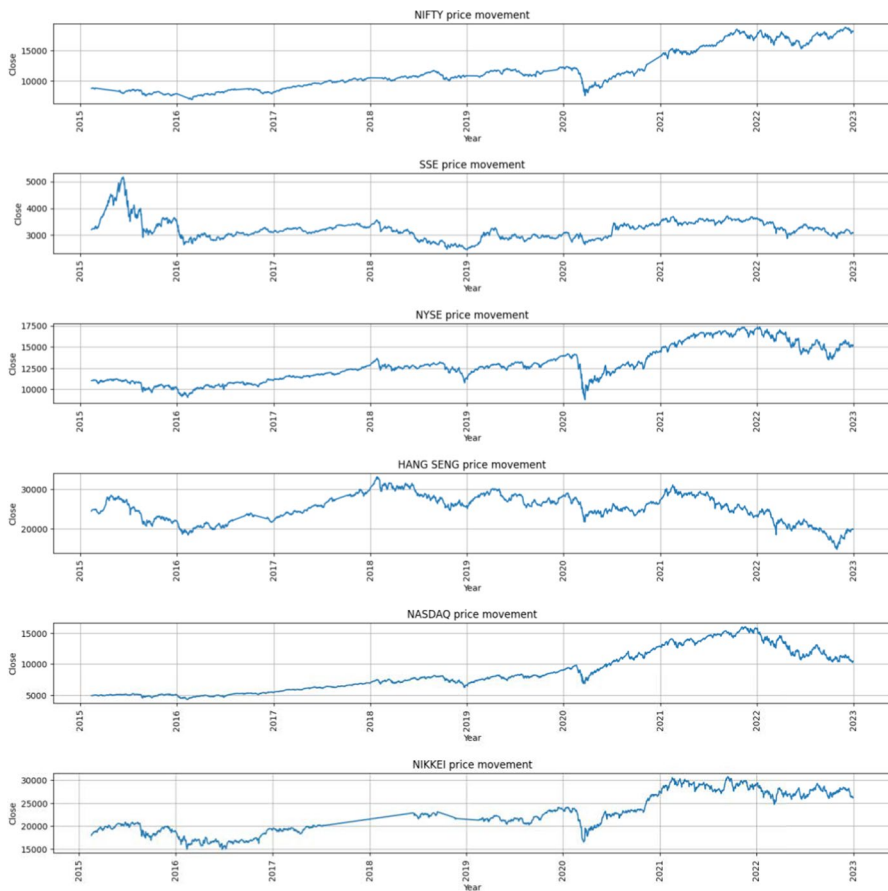


Fig. 3 Price movement of the selected stocks

4.3 Data Processing and Labeling

The first step in data processing involves the removal of any null values, which, in this case, were found to be absent. The weights are also computed considering the class imbalance, giving more weight to the minority class during training. As part of the data labeling process, this study adopts a similar methodology, as previously employed in Khodae et al. (2022); Liu, (2018), that utilizes a binary classification scheme based on the observed trends of short-term market movements. Specifically, the labeling assigns a value of 0 to denote a downward trend in Eq. (10) and 1 to indicate an upward trend as given in Eq. (11).

$$label_k = 0 \text{ for } close_{t+k} \leq close_t \quad (10)$$

$$label_k = 1 \text{ for } close_{t+k} > close_t \quad (11)$$

The labeling methodology represents the closing price on the current day $close_t$, and the closing price on a future day as $close_{t+k}$, where k is the number of days ahead, and $label_k$ is the binary label for k -day ahead prediction (1 day, 3 days, 7 days, or 10 days).

5 Experiments, Result Analysis, and Validation

To prove the effectiveness of the CAG model, this method is compared with CNN, LSTM, GRU, and CNN-GRU using the same training set and test set data.

All of these methods are implemented in Python and Keras, an open-source machine-learning library built on TensorFlow. The experiments are conducted within the Google Colab cloud environment, utilizing hardware specifications including an Intel(R) Xeon(R) CPU @ 2.30 GHz, 12 GB of RAM, and running on Windows 10. For assessing both the CAG model and the benchmark models utilized in this study, we evaluate their classification performance and profitability within a simulated trading scenario. The experiments are conducted with an epoch count of 50, utilizing the Adam optimizer, binary cross-entropy loss, a batch size of 64, and a learning rate set at 0.001.

5.1 Performance Metrics

The comprehensive performance of the proposed CAG model is evaluated using two critical evaluation criteria: classification performance and a financial evaluation when compared against the benchmark models. The classification report succinctly summarizes the model's performance in differentiating between the *buy* and *sell* class labels. Meanwhile, the financial evaluation examines the models in terms of profitability, vividly depicting the model's predictive performance from a real-world financial scenario, wherein stocks are bought and sold with the actual stock data. The classification metrics include recall, precision, f1-Score, and accuracy. For a more detailed explanation of these performance metrics and their application in evaluating the model's performance, one can refer to Jiaju & Wei (Miao & Zhu, 2022).

The profitability and risk metrics used herein include return on investment (ROI) (Friedlob & Plewa, 1996), annualized return (AR) (Fama, 1990), and maximum drawdown (MDD) (Magdon-Ismail & Atiya, 2004).

Metric	Formula	Description
Annualized return (AR)	$1 + ROI^{\frac{1}{Noofyears}} - 1$	Represents the compound rate of return per year, considering the investment holding period
Maximum drawdown (MDD)	$\frac{Peak_{value} - Trough_{value}}{Peak_{value}} \times 100$	Measures the maximum percentage loss from the highest peak to the lowest trough during a specific investment period, assessing the model's risk and potential for substantial losses
Return on investment (ROI)	$\frac{Final_{portfolio} - Initial_{portfolio}}{Initial_{portfolio}} \times 100$	Calculates the percentage of profit or loss relative to the initial investment, quantifying the profitability of the model's predictions

The overall success of the model depends on its ability to accurately predict each class, to avoid generating false *buy* and *sell* decisions. It is important because the *sell* class labels appear much more than the *buy* class labels in all of the datasets except for the NIFTY dataset, where an upward price movement is observed. This makes it very difficult for the model to identify the less predominant *buy* class, especially as the prediction interval increases. Each trade is initiated using the trading strategy outlined in Algorithm 2 where a *hold* strategy is introduced. This ensures the model tries to sell at a higher close price than it bought, but also uses a stop loss in cases where the price movement is constantly moving in a bearish trend.

Algorithm 2: Proposed Trading Strategy

```

1.   generate signals based on the predicted binary values:
2.   iterate over the range of y_pred_binary:
3.       if y_pred_binary[i] equals 1, set signals.iloc[i+time_steps] to 1
4.       else if y_pred_binary[i] equals 0, set signals.iloc[i+time_steps] to -1
5.   concatenate the 'close' column of x_test and the 'signals' dataframe to create df_signals.
6.   define initial portfolio balance and trade execution price:
7.       initial_balance = 100000
8.       execution_price = x_test['close']
9.   generate entry and exit signals:
10.      entry_signals = df_signals[df_signals['signal'] == 1]
11.      exit_signals = df_signals[df_signals['signal'] == -1]
12.   initialize variables and data structures:
13.      portfolio_balance = initial_balance, total_buys = 0, total_sells = 0, profits_losses = []
14.      entry_exit_dates = [], balance_values = []
15.   define the hold period:
16.      hold_period = 1
17.   iterate over the entry signals:
18.   for i in range(len(entry_signals)):
19.       entry_index = entry_signals.index[i]
20.       entry_price = execution_price[entry_index]
21.       entry_exit_dates.append(entry_index)
22.       total_buys += 1
23.   find the corresponding exit signal after the hold period:
24.       exit_index = none
25.   for j in range(i + 1, min(i + hold_period + 1, len(exit_signals))):
26.       if exit_signals.index[j] > entry_index:
27.           exit_index = exit_signals.index[j]
28.       break
29.   check if an exit signal is found:
30.   if exit_index is not none:
31.       exit_price = execution_price[exit_index]
32.       entry_exit_dates.append(exit_index)
33.       total_sells += 1
34.   calculate profit/loss:
35.       profit_loss = (exit_price - entry_price) / entry_price
36.       profits_losses.append(profit_loss)
37.   update portfolio balance:
38.       portfolio_balance *= (1 + profit_loss)
39.       balance_values.append(portfolio_balance)
40.   convert balance_values to a numpy array:
41.       final_balance = np.array(balance_values)

```

5.2 Prediction Results and Discussions

The results confirm the effectiveness of the proposed CAG model in distinguishing between the *buy* and *sell* classes across all the indices taken in this experiment. The classification report, presented in Table 2, highlights the classification results to analyze the accuracy of the predictions made by each model on each prediction interval (1 day, 3 days, 7 days, and 10 days).

It is observed the classification results decrease as the prediction interval increases across all indexes, this is because it becomes more difficult to predict longer intervals than immediate ones. On the BSE index, the CAG model achieves an average accuracy of 0.89 across different prediction intervals, indicating its efficiency in accurately predicting each class label. Alongside, the CNN and CNN-GRU models also demonstrate strong performance, with an average accuracy of 0.82. The inclusion of the AM model becomes evident even with the hybrid model of CNN-GRU using the same parameters. The individual benchmark models especially the CNN model also performs optimally across the 1 and 3 days intervals. Similarly, on the HSI index, the proposed CAG model surpasses other models with an average accuracy of 0.88, outperforming the closest model, CNN-GRU, by 0.01 in terms of accuracy. On the IXIC index, the proposed CAG model achieves an average accuracy of 0.89, demonstrating its robust predictive capabilities. Furthermore, in the NIFTY50 index, the CAG model exhibits outstanding performance, with an average accuracy of 0.88, surpassing CNN-GRU by 0.03 in accuracy. As indicated in the previous section, the NIFTY index is the only one which has a continuous upward trend movement, making it slightly different from the others. Additionally, the class imbalance is much closer in this index as compared to the other datasets. On the N-225 index, the proposed CAG model shows an average accuracy of 0.90, indicating its ability to accurately predict stock market movements. The CNN model also achieves an average accuracy of 0.85 in the same index. Similarly, on the SSE index, both the proposed CAG model and the CNN-GRU model achieve an average accuracy of 0.89 and 0.82, respectively, showing their effectiveness in predicting market movements.

The predictive performance of the proposed CAG model exhibits remarkable efficiency in accurately predicting both the *buy* and *sell* classes, surpassing the performance of the other models examined in this study. This is particularly noteworthy considering the inherent class imbalance in the datasets, where most instances are associated with a downward market trend. A pivotal element contributing to the superior predictive performance of the proposed CAG model is the incorporation of the AM, which adds a significant advantage in improving learning of the datasets during training. This ensures that the CAG model applies appropriate attention on the weightage to each class, effectively mitigating the risk of false *buy* and *sell* labels. Figure 4 displays the accuracy of the proposed CAG model against other benchmark models.

The execution time of each model was analyzed and presented in Fig. 5. Notably, the proposed CAG model demonstrated the overall shortest execution time among all the models, making it computationally efficient for real-time stock market trend prediction. As expected, the execution time increased as the prediction time interval

Table 2 Classification Report of proposed CAG Model against other benchmark models

MOD- ELS	1 Day			3 Day			7 Day			10-DAYS		
	Preci- sion	Recall	F1-Score	Accu- racy	Preci- sion	Recall	F1-Score	Accu- racy	Preci- sion	Recall	F1-Score	Accuracy
BSE index												
CNN	0.95	0.94	0.94	0.94	0.94	0.94	0.94	0.94	0.84	0.83	0.83	0.78
LSTM	0.96	0.96	0.96	0.96	0.79	0.78	0.78	0.78	0.82	0.81	0.81	0.75
GRU	0.94	0.94	0.94	0.94	0.91	0.90	0.90	0.90	0.83	0.82	0.82	0.75
CNN- GRU	0.92	0.92	0.92	0.92	0.93	0.93	0.93	0.93	0.80	0.79	0.79	0.72
CAG (pro- posed)	0.97	0.97	0.97	0.97	0.95	0.95	0.95	0.95	0.85	0.84	0.84	0.80
HSI index												
CNN	0.93	0.90	0.90	0.91	0.85	0.83	0.84	0.85	0.85	0.85	0.85	0.76
LSTM	0.68	0.67	0.65	0.66	0.78	0.74	0.69	0.69	0.82	0.83	0.83	0.72
GRU	0.74	0.73	0.73	0.73	0.82	0.84	0.83	0.84	0.85	0.80	0.81	0.70
CNN- GRU	0.90	0.88	0.88	0.88	0.84	0.84	0.84	0.84	0.84	0.84	0.84	0.72
CAG (pro- posed)	0.98	0.98	0.98	0.98	0.88	0.88	0.88	0.88	0.86	0.85	0.85	0.79
IXIC index												
CNN	0.93	0.94	0.94	0.94	0.88	0.87	0.88	0.88	0.87	0.86	0.87	0.79
LSTM	0.92	0.92	0.92	0.92	0.87	0.87	0.86	0.87	0.84	0.84	0.84	0.81
GRU	0.87	0.86	0.86	0.87	0.84	0.81	0.82	0.83	0.86	0.85	0.85	0.78
CNN- GRU	0.96	0.95	0.96	0.96	0.89	0.88	0.88	0.89	0.84	0.80	0.81	0.80

Table 2 (continued)

MOD- ELS	1 Day			3 Day			7 Day			10-DAYS		
	Preci- sion	Recall	F1-Score	Accu- racy	Preci- sion	Recall	F1-Score	Accu- racy	Preci- sion	Recall	F1-Score	Accuracy
GRU	0.81	0.81	0.81	0.81	0.86	0.86	0.86	0.86	0.74	0.73	0.74	0.68
CNN- GRU	0.92	0.92	0.92	0.92	0.82	0.81	0.81	0.81	0.83	0.83	0.83	0.72
CAG (pro- posed)	0.96	0.96	0.96	0.96	0.89	0.88	0.88	0.88	0.87	0.87	0.87	0.85

(number of days) increased, also indicating the inherent complexity of longer-term predictions.

5.3 Trend Analysis Strategy

In addition to the classification results presented and discussed in Sect. 5.2, the predicted results generated by all the models were employed to simulate transactions using a trading strategy outlined in Algorithm 2. The quantified results presented in Table 3 corroborate the exceptional performance of the proposed CAG model across the diverse indices considered in this study. This presents the number of buy trends and sell trends observed in all the trend analysis strategies and the proposed CAG model at the 10 days interval. It also outlines the trend observed using only technical indicators used by traditional market participants, using average directional index (ADX), rate of change (ROC), and moving average convergence divergence (MACD), and the profitability of initiating trades using these indicators alone.

As observed from Table 3, these technical indicators are more volatile than the DL models, which increases the chances of initiating false buy and sell signals. MDD, in particular, measures the volatility of the model when employed with the proposed trading strategy, where a lower MDD value indicates a less risky model. Notably, the CAG model demonstrated commendable performance with lower MDD values compared to other benchmark models in most cases. Across the BSE, HSI, IXIC, NIFTY, N225, and SSE indexes, the CAG model yielded remarkable ROIs of 11.30%, 17.68%, 18.02%, 15.84%, 18.68%, and 14.09%, respectively. The AR metric provides insights into the models' average returns per trend. Also, the CAG model displayed outstanding performance with consistently higher AR values across all stock market indexes. It achieved AR values of 6.67%, 15.97%, 15.19%, 15.53%, 19.45%, and 15.56% for the BSE, HSI, IXIC, NIFTY, N225, and SSE indexes, respectively.

The performance of the CAG model was also compared against the buy & hold (B&H) strategy, where the trader simply purchases and holds the index data for the duration of the test data. The return on investment (ROI) for the B&H strategy yielded unfavorable results of -11.56%, -23.07%, -31.26%, 7.96%, -8.40%, and -12.90% for the BSE, HSI, IXIC, NIFTY, N225, and SSE indexes, respectively. In stark contrast, the CAG model significantly outperforms the B&H strategy, attaining superior ROI values of 19.78%, 13.69%, 5.64%, 16.18%, 15.40%, and 0.93% across the respective indexes.

The visualization depicted in Fig. 6 showcases the buy and sell signals generated by the proposed CAG model across a diverse range of indexes. This visual representation provides a tangible glimpse into the model's predictive capabilities and its ability to identify opportune moments to make buy and sell orders. Figures 7, 8, 9, 10 gives a graphical representation of the buy and sell signals by the compared models used in this study. These findings unequivocally demonstrate the remarkable performance and profitability of the CAG model, even in the face of challenging market conditions, and underscore its potential as a valuable tool for stock market trading.

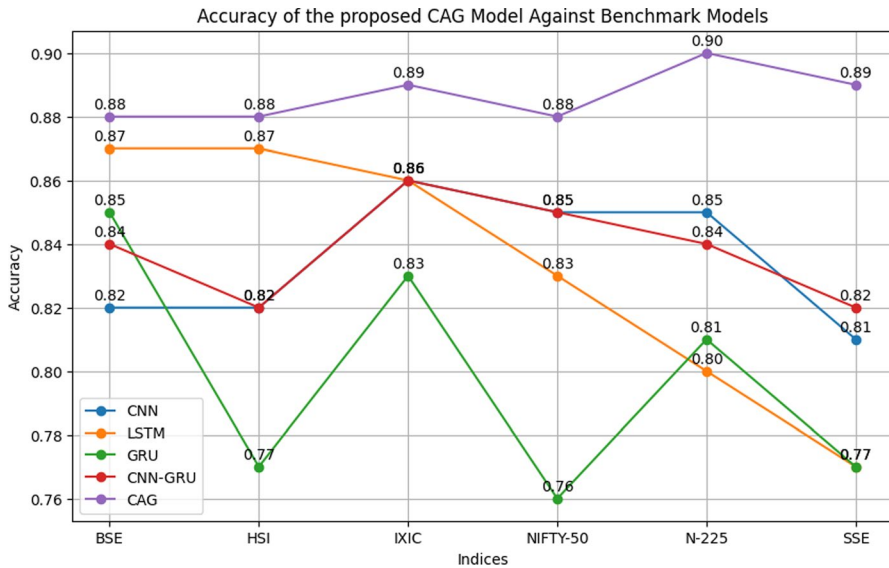


Fig. 4 Accuracy of proposed CAG Model against other benchmark models

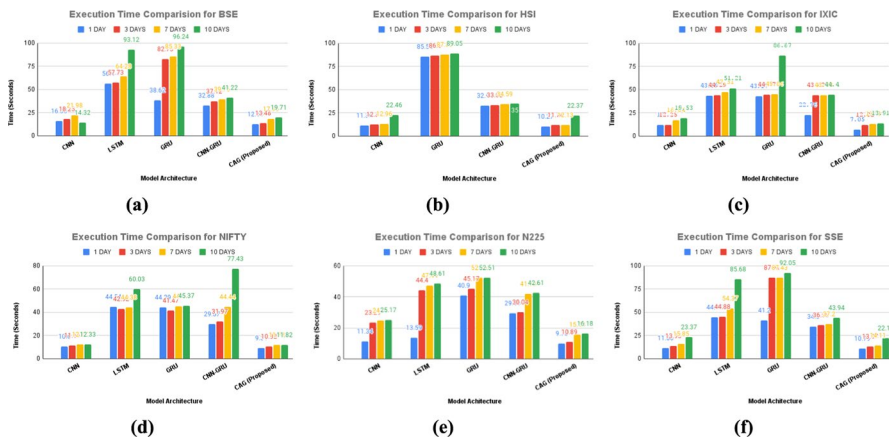


Fig. 5 Comparison of execution time (in seconds) for different days for CNN, LSTM, GRU, CNN-GRU, and CAG for all indexes

5.4 Statistical Validation

To support our findings, we performed a statistical validation of the CAG model and the other benchmark model used in the study using the wilcoxon signed rank test (Woolson, 2007) similar to the study by Jothimani & Surendra (Jothimani & Yadav, 2019), summarized in Table 4.

Table 3 Financial Evaluation of proposed CAG Model against other benchmark models

Methods compared	No. of trends	BSE	HSI	IXIC	NIFTY	N225	SSE
Actual trends observed	Buy trends	1075	1023	1110	1067	1062	1030
	Sell trends	939	951	904	909	913	915
ROC	Buy trends	1219	937	1287	1064	932	1013
	Sell trends	759	915	698	653	659	878
	ROI (%)	5.68	− 0.67	− 12.12	4.26	− 7.98	− 13.42
ADX	Buy trends	784	781	849	721	556	978
	Sell trends	221	154	151	120	185	144
	ROI (%)	7.51	− 1.86	− 9.14	6.19	5.07	0.64
MACD	Buy trends	77	68	80	67	70	70
	Sell trends	77	68	80	67	70	70
	ROI (%)	3.51	− 19.00	− 13.87	5.61	− 15.07	− 3.58
CNN	Buy trends	86	94	86	68	70	63
	Sell trends	85	94	86	67	69	63
	ROI (%)	4.05	9.82	4.92	7.20	14.79	− 7.41
	AR	3.58	6.94	4.15	7.06	15.40	− 6.48
	MDD	9.84	11.60	12.22	9.19	8.14	9.92
LSTM	Buy trends	81	52	96	70	52	94
	Sell trends	81	51	96	70	51	94
	ROI (%)	3.45	6.01	3.05	5.20	3.57	2.04
	AR	2.91	4.46	2.57	5.09	3.72	1.78
	MDD	18.11	10.54	17.88	10.37	11.98	11.58
GRU	Buy trends	82	81	99	70	51	78
	Sell trends	81	80	99	70	50	78
	ROI (%)	7.25	− 13.74	14.54	0.19	8.38	− 1.03
	AR	6.11	− 8.67	12.25	0.19	8.72	− 0.90
	MDD	11.16	33.17	19.08	11.82	14.64	13.16
CNN-GRU	Buy trends	86	88	93	59	51	61
	Sell trends	85	87	93	58	50	60
	ROI (%)	4.25	12.79	14.36	7.51	11.27	7.66
	AR	3.58	11.55	12.11	7.36	12.50	6.70
	MDD	9.84	8.44	19.96	13.38	7.77	10.97
CAG (proposed)	Buy trends	83	86	89	59	46	71
	Sell trends	83	85	89	58	45	70
	ROI (%)	11.30	17.68	18.02	15.84	18.68	14.09
	AR	6.67	15.97	15.19	15.53	19.45	15.56
	MDD	5.62	2.72	8.40	7.37	5.69	9.79

- *Step 1* Formulate the null hypothesis and alternative hypothesis: the null hypothesis is that there is no significant difference between the accuracies of the models. The alternative hypothesis is that there is a significant difference between the accuracies of the models.

- *Step 2* Calculate the signed-rank: for each model, calculate the signed-rank for each dataset. The signed rank is the rank of the difference between the model's accuracy on the dataset and the average accuracy of all models.
- *Step 3* Calculate the wilcoxon signed rank test statistic: the wilcoxon signed rank test statistic is the sum of the positive signed ranks. In this case, the Wilcoxon signed rank test statistic is 35.
- *Step 4* Calculate the p value: the p value is the probability of getting a wilcoxon signed rank test statistic as extreme or more extreme than the one you observed, assuming the null hypothesis is true. In this case, the p -value is 0.0003: which is less than 0.05, so the null hypothesis is rejected indicating there is a significant difference between the accuracies of the models.

5.5 Comparison Results on Different Methods

Building upon prior sections and the experiments conducted in this study, a comparative analysis evaluated the performance of the CAG model against three prominent techniques used in similar studies. This analysis included AT-LSTM and AT-GRU (Jinghua et al., 2020) (Zhao et al., 2021), ANN + IFS (Binita & Tripti, 2023) (Loo et al., 2023), and GA-TDDPL-CNN-LSTM (Wei Chuan Loo et al., 2023) (Kumari & Swarnkar, 2023). All models were trained and tested on identical datasets with consistent feature sets, ensuring a fair and objective comparison. Importantly, these prior studies primarily focused on 1 day-ahead market trend predictions. Table 5 demonstrates the CAG model's superior performance across both feature sets, outperforming both AT-LSTM and AT-GRU. The CAG model gets an accuracy of 0.92 as compared to 0.56 and 0.52 for the AT-LSTM and AT-GRU respectively.

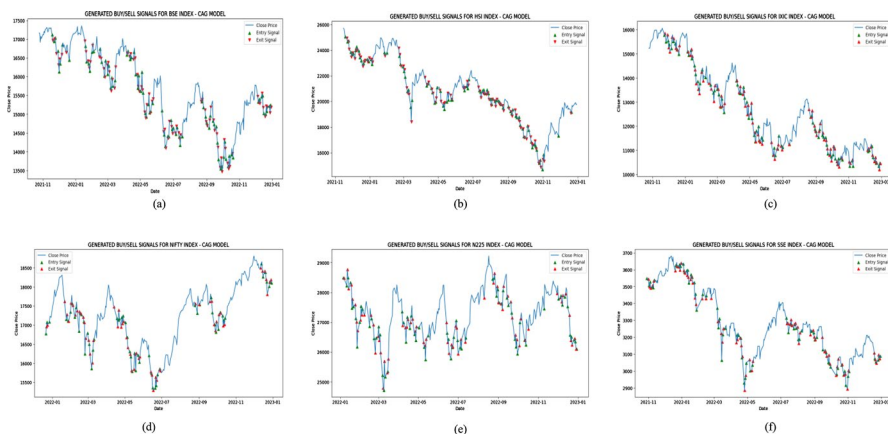


Fig. 6 Generated buy and sell signals of the proposed CAG Model

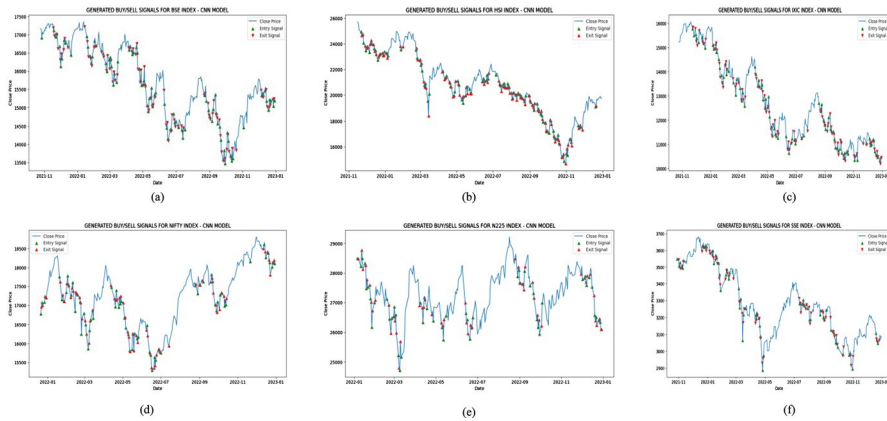


Fig. 7 Generated buy and sell signals of CNN Model

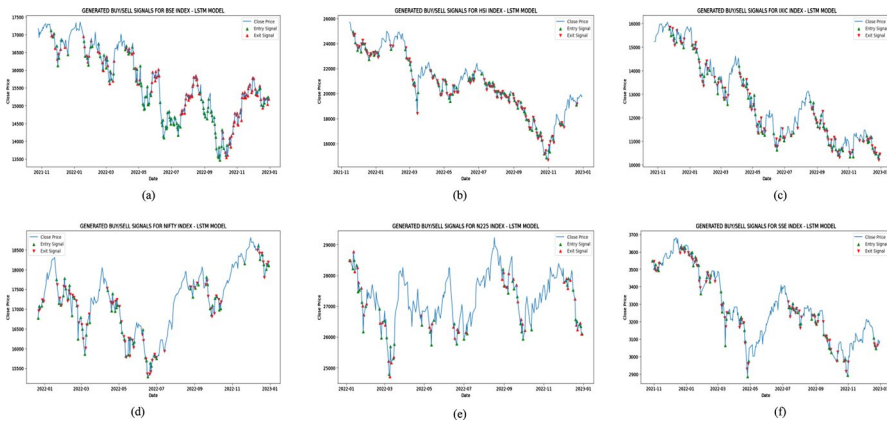


Fig. 8 Generated buy and sell signals of LSTM Model

Additionally, the CAG model was also compared with the ANN+IFS hybrid model (Kumari & Swarnkar, 2023) using similar evaluation metrics. Table 6 reveals that the CAG model exhibits superior classification performance, further confirming its effectiveness in predicting market trends. Overall, the CAG model gets an average accuracy of 0.97 and the ANN+IFS gets an average accuracy of 0.87 predicting a 1 day market trend. Finally, the CAG model is also compared against the GA-TDDPL-CNN-LSTM [50] using the S&P500 data from the referenced study. The results assert its generalizability and robustness across different markets and data distributions (Table 7).

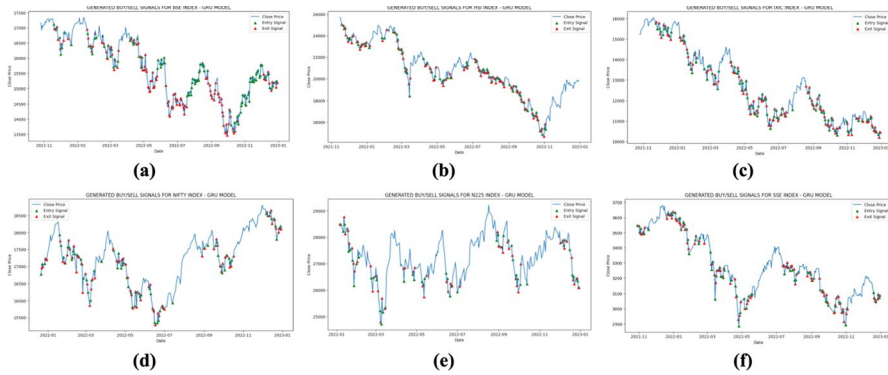


Fig. 9 Generated buy and sell signals of GRU Model

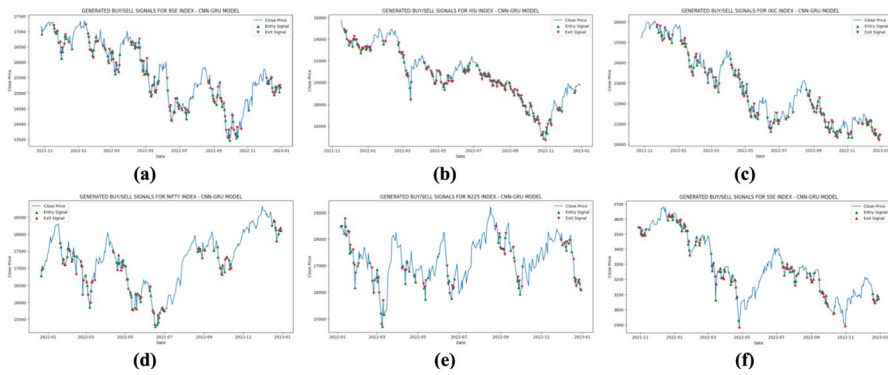


Fig. 10 Generated buy and sell signals of the CNN-GRU Model

Table 4 Statistical Validation of proposed CAG Model against other benchmark models

Model	Average accuracy	Signed rank	Wilcoxon signed rank test statistic	<i>p</i> value	Rank
CAG (proposed)	0.88	10	35	0.0003	1
CNN-GRU	0.86	8	32	0.0009	2
CNN	0.84	6	24	0.0028	3
LSTM	0.83	4	16	0.0056	4
GRU	0.81	2	8	0.0113	5

Table 5 Performance evaluation of the CAG model on 60300.HK

Index	Model	Accuracy	Precision	Recall	F1-score
60300.HK	AT-LSTM	0.56	–	–	0.54
	AT-GRU	0.55	–	–	0.56
	CAG	0.92	0.93	0.92	0.92

Table 6 Performance evaluation of the CAG model on similar indices

Index	Model	Accuracy	Precision	Recall	F1-score
BSE	ANN+IFS	0.79	0.89	0.79	0.81
	CAG	0.97	0.97	0.97	0.97
HSI	ANN+IFS	0.90	0.85	0.81	0.82
	CAG	0.98	0.98	0.98	0.98
NIFTY50	ANN+IFS	0.89	0.76	0.81	0.78
	CAG	0.98	0.98	0.98	0.98
N225	ANN+IFS	0.90	0.83	0.80	0.82
	CAG	0.98	0.98	0.98	0.98
SSE	ANN+IFS	0.90	0.84	0.85	0.84
	CAG	0.96	0.96	0.96	0.96

6 Conclusion and Future Directions

In this study, we propose a novel hybrid approach called CAG that combines the strengths of CNN, AM, and GRU to predict stock market movements. We conducted experiments to evaluate the efficiency of the proposed model in predicting the *buy* and *sell* class labels. Six different indices (BSE, HSI, IXIC, NIFTY, N225, and SSE) were used, and the data was preprocessed, labeled, and appropriately weighted to account for the movements of the datasets. This was done to help the model learn more efficiently for the minority class which in this study was the *sell* label. We conducted a comprehensive experimental analysis with benchmark models, and the proposed CAG model consistently outperformed other models in terms of accuracy, precision, recall, and f1-score. The predicted labels were used to simulate a real-world market scenario, where each model would trigger or initiate trading decisions. The proposed CAG model demonstrated exceptional predictive capabilities across various indices, achieving a remarkable average accuracy of 88.53%, an MDD of 6.59%, an average ROI of 13.58%, and an AR of 14.72% across all the indices used in this study. A statistical validation using the wilcoxon signed rank test further supported the significance of the CAG model's performance over the benchmark models. Overall, the proposed CAG model showcased its potential in accurately predicting stock market trends, providing valuable insights and predictive performance for investors and traders.

Future research on the CAG model could explore its performance using real-time trading data. This would provide insights into how well the model captures

Table 7 Performance evaluation of the CAG model on S&P 500

Index	Model	Accuracy	Precision	Recall	F1-score
S&P 500	GA-TDDPL-CNN-LSTM	0.80	0.79	0.89	–
	CAG	0.91	0.91	0.90	0.91

market dynamics, as well as its profitability when transaction and slippage fees are taken into account. Additionally, researchers could apply the CAG model to high-frequency data and optimize it as new data becomes available.

References

- Banik, S., Sharma, N., Mangla, M., Mohanty, S. N., & Shitharth, S. (2022). LSTM based decision support system for swing trading in stock market. *Knowledge-Based Systems*. <https://doi.org/10.1016/j.knosys.2021.107994>
- Borovkova, S., & Tsiamas, I. (2019). An ensemble of lstm neural networks for high-frequency stock market classification. *Journal of Forecasting*, 38, 600–619. <https://doi.org/10.1002/for.2585>
- Chatzis, S. P., Siakoulis, V., Petropoulos, A., Stavroulakis, E., & Vlachogiannakis, N. (2018). Forecasting stock market crisis events using deep and statistical machine learning techniques. *Expert Systems with Applications*, 112, 353–371. <https://doi.org/10.1016/j.eswa.2018.06.032>
- Chen, Y., Lin, Z., Zhao, X., Wang, G., & Yanfeng, Gu. (2014). Deep learning-based classification of hyperspectral data. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 7(6), 2094–2107.
- Chung, Junyoung, Caglar, G., KyungHyun, C., & Yoshua, B. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint [arXiv:1412.3555](https://arxiv.org/abs/1412.3555)
- Engle, R. (2001). GARCH 101: The use of ARCH/GARCH models in applied econometrics. *Journal of Economic Perspectives*, 15(4), 157–168.
- Fama, E. F. (1990). Stock returns, expected returns, and real activity. *The Journal of Finance*, 45(4), 1089–1108.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669. <https://doi.org/10.1016/j.ejor.2017.11.054>
- Fogel, S., Averbuch-Elor, H., Cohen, S., Mazor, S., & Litman, R. (2020). Scrabblegan: Semi-supervised varying length handwritten text generation. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 4324–4333
- Friedlob, G. T., & Plewa, F. J., Jr. (1996). *Understanding return on investment*. Wiley.
- Gandmal, D. P., & Kumar, K. (2019). Systematic analysis and review of stock market prediction techniques. *Computer Science Review*. <https://doi.org/10.1016/j.cosrev.2019.08.001>
- Hu, Q., Zhang, R., & Zhou, Y. (2016). Transfer learning for short-term wind speed prediction with deep neural networks. *Renewable Energy*, 85, 83–95.
- Huang, B., Huan, Y., Xu, L. D., Zheng, L., & Zou, Z. (2019). Automated trading systems statistical and machine learning methods and hardware implementation: A survey. *Enterprise Information Systems*, 13, 132–144. <https://doi.org/10.1080/17517575.2018.1493145>
- Istiake Sunny, M. A., Maswood, M. M. S., & Alharbi A. G. (2020). Deep Learning-Based Stock Price Prediction Using LSTM and Bi-Directional LSTM Model. In: 2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES). Pp 87–92. <https://doi.org/10.1109/NILES50944.2020.9257950>
- James, C., Van, H., & Parker, G. (1967). The random-walk theory: An empirical test. *Financial Analysts Journal*, 23, 87–92. <https://doi.org/10.2469/faj.v23.n6.87.17>

- Ji, S., Kim, J., & Im, H. (2019). A comparative study of bitcoin price prediction using deep learning. *Mathematics*. <https://doi.org/10.3390/math7100898>
- Jothimani, D., & Yadav, S. S. (2019). Stock trading decisions using ensemble-based forecasting models: A study of the Indian stock market. *Journal of Banking and Financial Technology*, 3, 113–129.
- Khodaei, P., Esfahanipour, A., & Taheri, H. M. (2022). Forecasting turning points in stock price by applying a novel hybrid cnn-lstm-resnet model fed by 2d segmented images. *Engineering Applications of Artificial Intelligence*. <https://doi.org/10.1016/j.engappai.2022.105464>
- Kim, T., & Kim, H. Y. (2019). Forecasting stock prices with a feature fusion LSTM-CNN model using different representations of the same data. *PLoS ONE*. <https://doi.org/10.1371/journal.pone.0212320>
- Kumari, B., & Swarnkar, T. (2023). Forecasting daily stock movement using a hybrid normalization based intersection feature selection and ANN. *Procedia Computer Science*, 218, 1424–1433. <https://doi.org/10.1016/j.procs.2023.01.121>
- LeCun, Y., & Bengio, Y. (1995). Convolutional networks for images, speech, and time series. *The Handbook of Brain Theory and Neural Networks*, 3361(10), 1995.
- Lei, K., Zhang, B., Li, Y., Yang, M., & Shen, Y. (2020). Time-driven feature aware jointly deep reinforcement learning for financial signal representation and algorithmic trading. *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswa.2019.112872>
- Li, S., Huang, X., Cheng, Z., Zou, W., & Yi, Y. (2023). AE-ACG: A novel deep learning-based method for stock price movement prediction. *Finance Research Letters*. <https://doi.org/10.1016/j.frl.2023.104304>
- Liu, H. (2018). Leveraging financial news for stock trend prediction with attention-based recurrent neural network. arXiv preprint [arXiv:1811.06173](https://arxiv.org/abs/1811.06173)
- Loo, W. C., Malik, E. F., Chew, X. Y., Khaw, K. W., Saha, S., Lee, M. H., & Al Akasheh, M. (2023). A proposed hybrid GA-TDDPL-CNN-LSTM architecture for stock trend prediction. *International Journal of Intelligent Systems and Applications in Engineering*, 11(3), 653–664.
- Magdon-Ismail, M., & Atiya, A. F. (2004). Maximum drawdown. *Risk Magazine*, 17(10), 99–102.
- Malkiel, G. B. (2003). The efficient market hypothesis and its critics. *Journal of Economic Perspectives*, 17, 59–82. <https://doi.org/10.1257/089533003321164958>
- Miao, J., & Zhu, W. (2022). Precision–recall curve (PRC) classification trees. *Evolutionary Intelligence*, 15, 1545–1569. <https://doi.org/10.1007/s12065-021-00565-2>
- Oscar, B., & Pomares-Quimbaya, A. (2020). Stock market movement forecast: A systematic review. *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswa.2020.113464>
- Rekha Das, S., Mishra, D., & Rout, M. (2019). Stock market prediction using firefly algorithm with evolutionary framework optimized feature reduction for oselm method. *Expert Systems with Applications*. <https://doi.org/10.1016/j.eswax.2019.100016>
- Sezer, O. B., & Ozbayoglu, A. M. (2018). Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach. *Applied Soft Computing*, 70, 525–538. <https://doi.org/10.1016/j.asoc.2018.04.024.18>
- Sharma, N., Soni, M., Kumar, S., Kumar, R., Deb, N., & Shrivastava, A. (2023). Supervised machine learning method for ontology-based financial decisions in the stock market. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(5), 1–24.
- Shen, J., & Shafiq, M. (2020). Short-term stock market price trend prediction using a comprehensive deep learning system. *Journal of Big Data*. <https://doi.org/10.1186/s40537-020-00333-6>
- Shun, C., & Lei, G. (2019). Exploring the attention mechanism in LSTM-based Hong Kong stock price movement prediction. *Quantitative Finance*, 19(9), 1507–1515. <https://doi.org/10.1080/14697688.2019.1622287>
- Sim, H. S., Kim, H. I., & Ahn, J. J. (2019). Is deep learning for image recognition applicable to stock market prediction? *Complexity*. <https://doi.org/10.1155/2019/4324878>
- Sober, E. (1981). The principle of parsimony. *The British Journal for the Philosophy of Science*, 32(2), 145–156.
- Terrence, H., Charles, M. J., & Albert, J. M. (2011). Does algorithmic trading improve liquidity? *The Journal of Finance*, 66, 1–33. <https://doi.org/10.1111/j.1540-6261.2010.01624.x>
- Treisman, A. M., & Gelade, G. (1980). A feature-integration theory of attention. *Cognitive Psychology*, 12(1), 97–136.
- Vargas, M. R., De Lima, B. S., & Evsukoff, A. G. (2017). Deep learning for stock market prediction from financial news articles. In: IEEE International Conference on Computational Intelligence and Virtual Environments for Measurement Systems and Applications (CIVEMSA). pp 60–65

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., (2017). Attention is all you need. *Advances in neural information processing systems*. p 30
- Wenjie, Lu., Li, J., Wang, J., & Qin, L. (2021). A CNN-BiLSTM-AM method for stock price prediction. *Neural Computing and Applications*, 33, 4741–4753.
- Woolson, R. F. (2007). *Wilcoxon signed-rank test* (pp. 1–3). Wiley.
- YahooFinance. At <https://finance.yahoo.com.CSV> file. Accessed March 2023
- Yang, S., Guo, H., & Li, J. (2022). CNN-GRUA-FC Stock Price Forecast Model Based on Multi-Factor Analysis. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 26(4), 600–608.
- Zhao, J., Zeng, D., Liang, S., et al. (2021). Prediction model for stock price trend based on recurrent neural network. *Journal of Ambient Intelligent Human Computing*, 12, 745–753. <https://doi.org/10.1007/s12652-020-02057-0>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Ibanga Kpereobong Friday¹ · Sarada Prasanna Pati¹ · Debahuti Mishra¹ · Pradeep Kumar Mallick² · Sachin Kumar³ 

✉ Sachin Kumar
sachinagnihotri16@gmail.com

Ibanga Kpereobong Friday
kpereib01@gmail.com

Sarada Prasanna Pati
saradapati@soa.ac.in

Debahuti Mishra
debahutimishra@soa.ac.in

Pradeep Kumar Mallick
pradeep.mallickfcs@kiit.ac.in

¹ Department of Computer Science and Engineering, Siksha 'O' Anusandhan (Deemed to Be) University, Bhubanaeshwar, Odisha, India

² School of Computer Engineering, KIIT Deemed to be University, Bhubanaeshwar, Odisha, India

³ Zaven and Sonia Akian College of Science and Engineering, American University Armenia, Yerevan, Armenia